mcp 服务接入说明

1. MCP 服务介绍

MCP （Model Context Protocol）定义了应用程序和 AI 模型之间交换上下文信息的方式。这使得开发者能够以一致的方式将各种数据源、工具和功能连接到 AI 模型（一个中间协议层），就像 USB-C 让不同设备能够通过相同的接口连接一样。MCP 的目标是创建一个通用标准，使 AI 应用程序的开发和集成变得更加简单和统一。

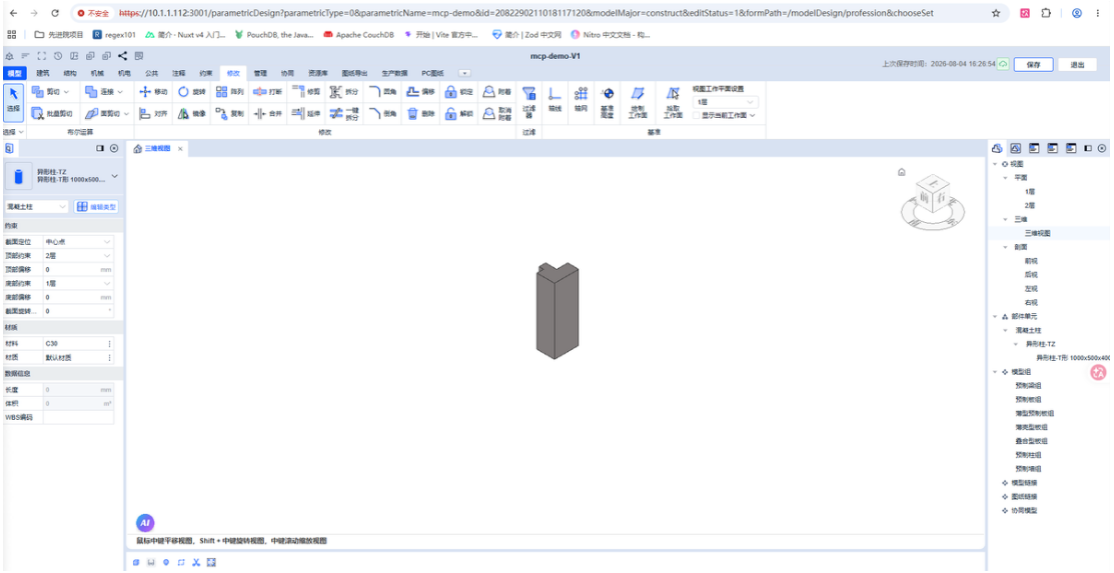
MCP Server 是让 AI 模型与外部工具、资源交互的核心组件。在 VScode 或 Codex 插件中配置 MCP Server，可以让 AI 直接访问文件系统、数据库、远程 API 等。而我们系统开发的 MCP Server 就是提供远程 API 使得 AI 大模型能够直接操作我们的模型编辑器以达到通过自然语言的方式来操作编辑器新增修改模型的目的。

2. 接入流程

当前 mcp 的服务能力支持两种接入方式：

1. **AI Agent 接入：**内嵌一个 MCP Server 提供给 AI Agent 的 Mcp Client 连接调用；
2. **Http Client 接入：**传统的 Http Restful API，直接接收 http 请求参数，根据参数执行相应动作；

4. 浏览器模型编辑器中自动创建了 'T 形混凝土柱'



验证完成，后续随着 Tools 工具的不断扩展 MCP 服务能完成的功能也会更加丰富。

2.2. HttpClient 接入

接入流程需要登录权限，如果要完成 2.1 章的新建 T 形混凝土柱 的流程需要依次调用如下接口：

2.2.1. 获取访问 token

2.2.1.1. 账号密码登录获取 token

登录接口是前端以前就存在的接口，使用用户名和密码获取访问凭证，这个凭证在接下来的所有接口请求时都需要在 Authorization 请求头中使用

需要自己处理好密码的加密工作，这里参考前端页面的登录逻辑。

接口：4.1.1 登录

2.2.1.2. oauth2 鉴权获取 token

oauth2 是一种开放授权协议，允许用户授权第三方应用访问其存储在服务提供商上的资源，而无需暴露用户的登录凭据。

oauth2 鉴权流程和 2.1.3.2.1 的激活流程一致，只不过 2.1.3.2.1 小结中由 vscode 自动处理的注册客户端、发起授权、使用授权 code 换取 access_token 的流程换成我们自己代码实现而已，获得 access_token 以后也是在 Authorization 请求头中使用。

假设我们用于接收授权码的回调地址为 http://localhost:5173，下面分别说明注册客户端、发起授权、使用授权 code 换取 access_token 的详细过程

2.2.1.2.1. 注册客户端

向平台注册客户端信息获取客户端唯一标识 client_id。

接口：4.1.4 oauth 注册客户端

参数示例：

```
{
    "client_name": "oauth2-demo-vue",
    "client_uri": "http://localhost:5173",
    "grant_types": [
        "authorization_code",
        "refresh_token"
    ],
    "response_types": [
        "code"
    ],
    "redirect_uris": [
        "http://localhost:5173"
    ],
    "scope": "cad:read cad:write",
    "token_endpoint_auth_method": "none",
    "application_type": "native"

}
```

返回示例：

```
{
  "client_id": "webcad-31b655bf-a1b9-44ea-962f-5e05e10617d9",
  "client_id_issued_at": 1786588623,
  "client_name": "oauth2-demo-vue",
  "redirect_uris": [
    "http://localhost:5173/"
  ],
  "token_endpoint_auth_method": "none",
  "grant_types": [
    "authorization_code",
    "refresh_token"
  ],
  "response_types": [
    "code"
  ],
  "scope": "cad:read cad:write"

}
```

2.2.1.2.2. 发起授权

调用平台授权连接弹出授权框获取用户授权，这里主要是要拼接授权 url 参数，授权参数说明：

完整示例：

https://10.1.1.112:3001/oauth/authorize?response_type=code&client_id=webcad-31b655bf-a1b9-44ea-962f-5e05e10617d9&redirect_uri=http%3A%2F%2Flocalhost%3A5173%2F&code_challenge=xQeQmPbd89TIErq11km9vrElsOW_Onqwd0EH5hWj42o&code_challenge_method=S256&scope=cad%3Aread+cad%3Awrite&resource=https%3A%2F%2Flocalhost%3A3000%2Fmcp&state=319f3c21-a638-4296-a43e-5bad84480c77

url：/oauth/authorize

method: Get

请求参数：

属性名	说明
client_id	客户端 ID，由上一小节的注册接口返回
response_type	授权类型，code 表示 Authorization Code Flow（授权码流程
redirect_uri	授权成功后跳转的地址，必须和注册时一致
code_challenge	PKCE 挑战码，base64url(SHA256(code_verifier))
code_challenge_method	PKCE 挑战码算法，S256 表示 SHA-256
scope	请求的权限范围，空格分隔

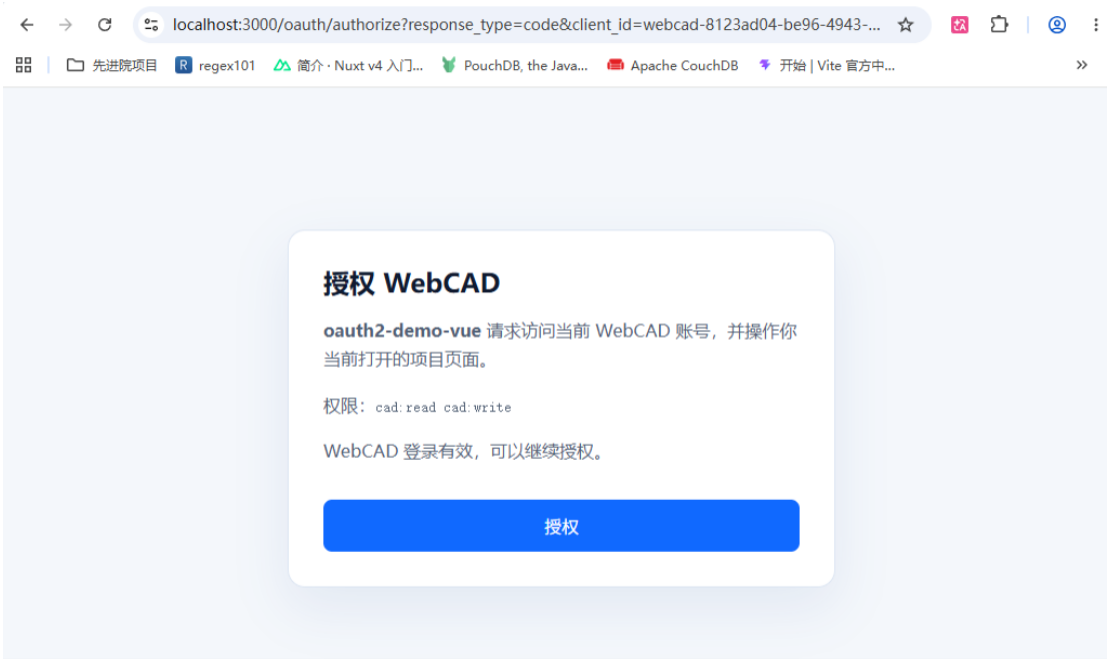
resource	目标资源服务器（RFC 8707）
state	防 CSRF 随机值，回调时原样返回

注意：code_challenge 和 code_challenge_method 两个参数是用于防止授权码被截获后冒用。生成逻辑如下：

- 客户端生成随机字符串 code_verifier（43-128 位）
- 计算 code_challenge = base64url(SHA256(code_verifier))
- 授权时传 code_challenge
- 换 token 时传 code_verifier
- 服务端验证：SHA256(code_verifier) === code_challenge

关于PKCE鉴权的标准可以查阅官方资料进一步了解，具体实现可以参考 2.2.1.2.4 附件中的完整代码

页面效果：



2.2.1.2.3. 获取 access_token

上一小节用户同意授权后浏览器会重定向到我们 2.2.1.2.1 注册接口中配置的 redirect_uris 上，并且会自动带上授权 code，然后我们可以用这个 code 值去换取 access_token

回调地址样例：

http://localhost:5173/?code=327ZHERsDb_bo0SpHlzkEy629dKdxtOehe5XL7QvIm8&state=319f3c21-a638-4296-a43e-5bad84480c77

接口：4.1.5 oauth 获取 access_token

参数示例：


```
{
    "grant_type":"authorization_code",
    "client_id":"webcad-31b655bf-a1b9-44ea-962f-5e05e10617d9",
    "code":"327ZHERsDb_bo0SpHlzkEy629dKdxtOehe5XL7QvIm8",
    "redirect_uri":"http://localhost:5173/",

"code_verifier":"SmispP4MxlhiucK3HzVNTuUJ_VuI5UQnlTS-8T0onh8hl1Wrr7_znnywza1Nvybp",
    "resource":"https://10.1.1.112:3001//mcp"
 }
```


返回示例：


```
{
    "access_token":
"eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2tleSI6InVwc3RyZWFtOkpPdmFZQ1YxX2pxSVVIYk0yX2RXQW5JcmIwcnZQQUZwNnVmTFl4NE1zUG8iLCJ1c2VyX2lkIjoiMzAiLCJ1c2VybmFtZSI6Inh1eGlhb2h1aSIsInNjb3BlIjoiY2FkOnJlYWQgY2FkOndyaXRlIiwidG9rZW5fdXNlIjoiY2FkX2FnZW50X29hdXRoIiwiYXVkIjoiaHR0cHM6Ly9sb2NhbGhvc3Q6MzAwMC9tY3AiLCJpYXQiOjE3ODY1OTI1ODIsImV4cCI6MTc4NjU5NjE4Mn0.L_KfPSOfKYoAPJI4BCsdbXvR1fwwnPOV4j88E_hsu2xWRplhKjL3I1CxTAO6DcJZ7L_MO1qP_rHDQAMrzCC9Hg",
    "token_type": "Bearer",
    "expires_in": 3600,
    "refresh_token": "C6lTpVqBEh02Opc0YZHsKqQXPv9WC8R_PCSjWliVWpo",
    "scope": "cad:read cad:write"

}
```


2.2.2. 获取 session 连接

获取当前与服务器连接的客户端 session 列表;这些客户端 session 保存有登录用户的信息以及websocket通道信息，通过这个接口的返回结果可以获取到 sessionId ，然后以路径参数的方式（4.3 发送操作指令 url 中的 sessionId 占位符）向指定的客户端发指令做节点的增删改查等操作。

接口：4.1.2 获取 session 列表

2.2.3. 获取支持的节点类型列表

根据 2.2.2 获取的 sessionId 访问 4.1.3 发送操作指令，得到节点类型和描述列表。其中包括 type、title、description 等。指令为`cad.node.list_types`

接口: 4.1.3 发送操作指令

参数示例：

```
{"tool":"cad.node.list_types","args":{}}
```

2.2.4. 获取节点 schema

以 2.2.3 得到的节点类型为参数获取节点结构描述 schema，包括新增、编辑节点需要提供的参数值，参数类型等。后面针对节点进行的操作都需要基于这个结构描述 schema。指令为 `cad.node.describe_type`

接口：4.1.3 发送操作指令

参数示例：

```
{
    "tool": "cad.node.describe_type",
    "args": {
        "type": "structure.tConcretePillar"
    }

}
```

2.2.5. 新增/修改节点

根据 2.2.4 获得的节点结构描述中的 createSchema 属性配来置构建请求参数，发送新增指令新增节点，指令为 `cad.node.create`

接口：4.1.3 发送操作指令

参数示例：

```
{
    "tool": "cad.node.create",
    "args": {
        "type": "structure.tConcretePillar",
        "params": {
            "startPoint": {
                "x": 0,
                "y": 0,
                "z": 0
            },
            "endPoint": {
                "x": 0,
                "y": 0,
                "z": 3000
            },
            "lengthB1": 200,
            "lengthB2": 200,
            "lengthB3": 200,
            "lengthB4": 200,
            "lengthA1": 400,
            "lengthA2": 200,
            "lengthA3": 400,
            "baseType": 0,
            "concreteType": "ConcretePillar",
            "crosssectionRotationAngle": 0
        }
    }

}
```

2.2.6. 完整示例

用户名+密码完整示例：

cad-agent-create-node.mjs

oauth2 授权完整示例：

oauth2-demo.zip

修改 vite.config.ts 中的 API_TARGET 常量切换授权环境

3. 注意事项

3.1. 本地启动

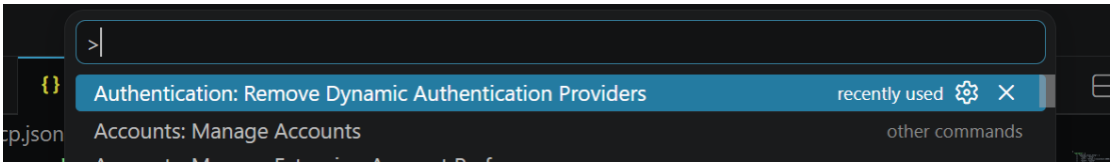
本地开发的时候除了启动前端 web 页面服务以外还需要提供 mcp 服务 API（如鉴权服务， mcp 基础注册服务等），所以启动脚本需要使用全量启动模式（即 `NUXT_WITH_LOCAL_API` 环境变量需要赋值为 1， 同时需要启动 CAD_AGENT），本地启动脚本为：

```
npm run devs:local:agent
```

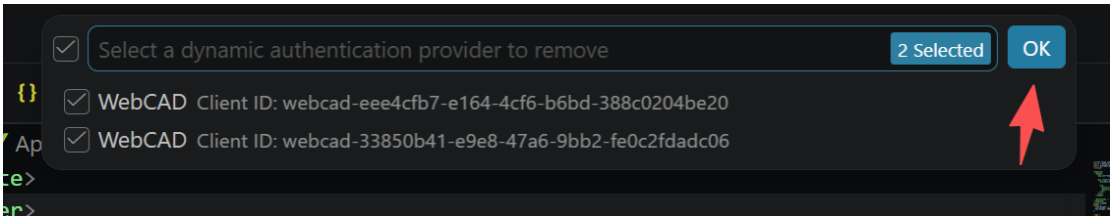
3.2. vscode 缓存清理

如果经常在测试环境和本地环境切换开发，有的时候 vscode 可能会串缓存数据导致切换的时候 mcp 服务激活失败，可以清除缓存重启 vscode 再连接，清除缓存操作如下：

1. 使用快捷键 `ctrl+shift+p`，选择 ‘Authentication:Remove Dynamic Authentication Providers’



2. 选中并清除 vscode 对应的 clientId 缓存：



3. 重启 vscode

3.3. 其他

对于 AI Agent 接入目前 vscode 没有排他的属性配置，需要确保每次只开启一个 mcp 服务。本地开发的时候停止 `webcad_112`，测试的时候停止 `webcad_local` ，两个都启动的时候由于它们提供的服务是一样的，AI Agent 会按照自己的规则选择一个执行，所以最终操作结果可能不确定：上面的 T 形混凝土柱 有可能会建到测试环境也有可能会建到本地。

虽然也可以通过完整的自然语言描述规避，比如每次指令都带上 ‘通过 webcad_local 的连接新建’ 等，但还是有一定风险的

总之对于我们的项目建议每次只激活一个环境的服务。

4. 附录

4.1. 接口说明

接口参数和属性比较多，当前不是纯粹的接口文档仅辅助说明调用流程，所以这里只列出关键属性。

4.1.1. 登录

url：/dev-api/auth/login

method: Post

请求参数：

属性名	说明	类型
username	登录名	String
password	登录密码	String
clientId	客户端编号	String
grantType	鉴权类型	string

返回值：

属性名	说明	类型
code	返回状态码	Number
msg	错误描述	String
data	结果数据	LoginResult

LoginResult：

属性名	说明	类型
access_token	访问凭证	String
expires_in	超时时间	Number

4.1.2. 获取 session 列表

url：/api/cad-agent/sessions

method: Get

请求参数：无
返回值：

属性名	说明	类型
sessions	session 列表	Session[]

Session:

属性名	说明	类型
id	唯一标识	String
userId	对应用户 Id	String
userName	用户名	String
pageUrl	session 对应的页面路径	String
documentId	模型 Id	String
documentName	模型名称	String
clientName	客户端名称	String

4.1.3. 发送操作指令

url：/api/cad-agent/sessions/{sessionId}/calls
method: Post
请求参数：

属性名	说明	类型
tool	操作指令	**Tool**
args	附加参数	Object

返回值：

属性名	说明	类型
ok	操作是否成功	Boolean
value	返回结果	Any（取决于 tool 的类型）

Tool：

指令 key	描述
cad.node.list_types	列出当前 WebCAD 引擎可由 Agent 创建的参数化 Node

	型摘要（type、semanticKey、title、description）。不含创新 Schema，如需完整 Schema 请用 cad.node.describe_按类型查询。
cad.node.describe_type	返回指定 Node 类型的 header 语义、创建 Schema 与更Schema。
cad.node.list_faces	列出宿主节点可用于 FaceRef 放置的拓扑面索引、世界坐法向和语义角色。
cad.node.create	使用完整 JSON 参数直接创建参数化 Node；不启动交互ICommand。
cad.node.get	读取指定参数化 Node 的 Agent 安全属性。
cad.node.update	按类型白名单更新参数化 Node，并触发历史记录与几何重
cad.node.delete	按 nodeId 删除指定参数化 Node。
cad.node.list	列出活动文档中所有 Node 的 id、type 与 semanticKey。
cad.assembly.list_types	列出可由 API/MCP 创建的组合构件类型摘要（type、semanticKey、title、description）。不含 Schema，如需完Schema 请用 cad.assembly.describe_type 按类型查询。
cad.assembly.describe_type	返回组合构件的创建与更新 Schema。
cad.assembly.list_schemes	列出组合构件可用的创建方案。创建预制板、墙、柱、梁钢若用户尚未指定方案名称或完整参数，必须先调用本工具并用户选择。
cad.assembly.create	按已注册构建器创建组合构件。创建预制板、墙、柱、梁钢不得传空 rebarData：用户已指定方案名称则传 schemeN用户已给出完整参数则传 rebarData；两者都没有时必须先方案并询问用户。
cad.assembly.get	读取组合构件的安全摘要。
cad.assembly.update	按组合构件构建器整体重建已有组合。
cad.document.undo	撤销活动文档最近一次修改。
cad.document.redo	重做活动文档最近一次撤销。
cad.document.save	保存当前活动文档。

随着开发深入会继续扩充

4.1.4. oauth 注册客户端

url：/oauth/register
method: Post
请求参数：

属性名	说明	类型
client_name	客户端描述	String
redirect_uris	回调地址	String[]
scope	授权范围	String
response_types	返回类型	String[]
grant_types	授权方式	String[]

返回值：

属性名	说明	类型
client_id	客户端 id	String
client_id_issued_at	生成时间	Number

4.1.5. oauth 获取 access_token

url：/oauth/token
method: Post
请求参数：

属性名	说明	类型
client_id	客户唯一标识	String
redirect_uri	回调地址	String
code	授权码	String
code_verifier	随机字符串，用户校验 code_challenge	String
grant_type	授权方式	String
resource	目标资源服务器（RFC 8707）	String

返回值：

属性名	说明	类型
access_token	授权 token	String

token_type	token 类型	String
expires_in	过期时间	Number
refresh_token	刷新 token	String
scope	请求的权限范围，空格分隔	String